

Laboratorio 4 - Análisis de Datos Geoespaciales

División de trabajo para entrega de avances - Ejercicios 1 al 4

CC3084 - Data Science | Semestre II 2026

Alcance del avance

Objetivo: completar únicamente los ejercicios 1 al 4 del laboratorio. Todo el proyecto se desarrollará en Python. Cada integrante mantiene un área técnica definida para evitar duplicación de trabajo y facilitar la integración posterior.

Contrato técnico del repositorio

Estructura fija: nadie debe cambiar nombres de carpetas, archivos, identificadores de lagos ni formato de fechas sin acuerdo del grupo.

```
lab4-datos-geoespaciales/
├── README.md
├── requirements.txt
├── .gitignore
├── .env.example
├── config.py
├── main.py
├── data/
│   ├── geojson/
│   │   ├── atitlan.geojson
│   │   └── amatitlan.geojson
│   └── rasters/
│       ├── atitlan/{YYYY-MM-DD}/
│       │   ├── ndvi.tif
│       │   ├── ndwi.tif
│       │   └── cyano.tif
│       └── amatitlan/{YYYY-MM-DD}/
│           ├── ndvi.tif
│           ├── ndwi.tif
│           └── cyano.tif
├── outputs/
│   ├── tables/temporal_summary.csv
│   └── figures/
│       ├── temporal_atitlan.png
│       └── temporal_amatitlan.png
└── src/
    ├── __init__.py
    ├── sentinel.py
    ├── indices.py
    └── temporal.py
```

Convenciones obligatorias: lagos = "atitlan" y "amatitlan"; fechas = YYYY-MM-DD; raster = ndvi.tif, ndwi.tif y cyano.tif. El CSV temporal debe contener exactamente: lake,date,cyano_mean,valid_pixels.

Persona 1 - Infraestructura geoespacial y Sentinel

Área permanente: estructura del proyecto, configuración, acceso a Sentinel-2 y obtención de datos.

Tareas para el avance

1. Crear el repositorio con exactamente la estructura definida en este documento.
2. Configurar el proyecto completamente en Python.
3. Crear requirements.txt, .gitignore y .env.example.
4. Crear config.py con las coordenadas oficiales, las 11 fechas de Atitlán, las 11 fechas de Amatitlán, rutas de GeoJSON y rutas base del proyecto.
5. Colocar atitlan.geojson y amatitlan.geojson dentro de data/geojson/.
6. Implementar autenticación con Sentinel Hub/API usando variables de entorno. Las credenciales no deben quedar versionadas.

7. Implementar `src/sentinel.py`.
8. Implementar una función reutilizable `request_product(lake, date, evalscript, output_path)` que reciba lago, fecha, script y ruta destino; ejecute la solicitud; guarde el raster y retorne su Path.
9. Usar únicamente las fechas oficiales proporcionadas por la guía.
10. Evitar descargar escenas completas cuando no sean necesarias.
11. Validar la descarga con al menos una fecha de Atitlán y una de Amatitlán.
12. Crear el esqueleto de `main.py` y un README mínimo con pasos de instalación y ejecución.

Entregables

- Repositorio inicial funcional.
- `config.py` completo y centralizado.
- `src/sentinel.py` funcional.
- `requirements.txt`, `.gitignore`, `.env.example` y `README.md`.
- GeoJSON ubicados en la ruta acordada.
- Prueba de descarga funcional para ambos lagos.
- Commits propios claramente identificables.

Criterio de aceptación: Persona 2 debe poder importar `request_product()` y obtener un raster sin reimplementar autenticación, coordenadas, fechas ni rutas.

Persona 2 - Procesamiento espectral

Área permanente: generación de NDVI, NDWI e índice de cianobacteria.

Tareas para el avance

13. Trabajar sobre la estructura creada por Persona 1 sin modificar nombres de carpetas ni contratos.
14. Importar lagos, fechas y rutas desde `config.py`. No duplicar coordenadas ni listas de fechas.
15. Crear `src/indices.py`.
16. Implementar NDVI usando B04 y B08.
17. Implementar NDWI usando B03 y B08.
18. Implementar el índice de cianobacteria usando el Cyano Detection Script oficial de Sentinel Hub.
19. Reutilizar `request_product()` de `src/sentinel.py` para cualquier solicitud a Sentinel.
20. Implementar `generate_indices()` para recorrer automáticamente los dos lagos y sus fechas configuradas.
21. Guardar los resultados únicamente como `data/rasters/{lake}/{date}/ndvi.tif`, `ndwi.tif` y `cyano.tif`.
22. Manejar valores NoData, NaN y píxeles inválidos.
23. Validar que los tres raster puedan abrirse y utilizarse correctamente.
24. Generar resultados suficientes para que Persona 3 pueda ejecutar el análisis temporal del avance.

Entregables

- `src/indices.py`.
- Función `generate_indices()`.
- Raster NDVI, NDWI y cianobacteria para las fechas procesadas.
- Resultados verificables para ambos lagos.
- Commits propios claramente identificables.

Criterio de aceptación: Persona 3 debe poder encontrar los productos en `data/rasters/{lake}/{date}/` sin conocer cómo fueron descargados ni cómo se calculó cada índice.

Persona 3 - Análisis temporal y visualización

Área permanente: estadísticas temporales, gráficas, identificación de picos e interpretación inicial.

Tareas para el avance

25. Trabajar únicamente con los raster generados por Persona 2 y la configuración central de `config.py`.
26. Crear `src/temporal.py`.
27. Leer automáticamente los lagos y fechas desde `config.py`.

28. Para cada lago y fecha, leer cyano.tif e ignorar NoData y píxeles inválidos.
29. Calcular cyano_mean y valid_pixels.
30. Generar outputs/tables/temporal_summary.csv con exactamente las columnas lake,date,cyano_mean,valid_pixels.
31. Ordenar las observaciones cronológicamente.
32. Generar outputs/figures/temporal_atitlan.png.
33. Generar outputs/figures/temporal_amatitlan.png.
34. Identificar valor máximo, valor mínimo, fecha del máximo y posibles picos de floración.
35. Tomar en cuenta al interpretar que Amatitlán 2026-02-07 tiene cobertura válida parcial aproximada de 57.1%.
36. Implementar run_temporal_analysis() para regenerar automáticamente tabla y gráficas.
37. Redactar una interpretación breve basada únicamente en los resultados observados.

Entregables

- src/temporal.py.
- Función run_temporal_analysis().
- outputs/tables/temporal_summary.csv.
- Gráfica temporal de Atitlán.
- Gráfica temporal de Amatitlán.
- Resumen de fechas críticas y picos.
- Interpretación inicial del ejercicio 4.
- Commits propios claramente identificables.

Criterio de aceptación: Con los raster existentes, run_temporal_analysis() debe regenerar automáticamente la tabla temporal y ambas gráficas.

Reglas de integración

- Persona 1 no calcula índices.
- Persona 2 no reimplementa acceso a Sentinel.
- Persona 3 no toca Sentinel ni las fórmulas espectrales.
- Nadie hardcodea rutas locales de su computadora.
- Las fechas y coordenadas existen una sola vez en config.py.
- No se agregan capas, patrones, frameworks o dependencias que no sean necesarias para los ejercicios 1 al 4.
- Para este avance no se implementan todavía análisis espacial, correlaciones, histogramas, estacionalidad ni comparación final entre lagos.

Resultado esperado del avance

- Conexión funcional con Sentinel-2.
- Datos obtenidos usando las fechas oficiales.
- NDVI, NDWI e índice de cianobacteria generados.
- Promedio de cianobacteria por lago y fecha.
- Gráfica temporal para cada lago.
- Identificación inicial de picos y fechas críticas.
- Interpretación inicial sustentada en los resultados.